

Remote Sensing Methods Reference

Every formula, constant and chart in the field remote sensing pipeline, written to be checked rather than run.

Almost nothing here is calibrated. The exceptions are CDL crop labels, soil armor and cover crop detection, tested against a 347-field-year Georgia farmer survey. Every constant fitted on this library was fitted on eleven fields. Section 10 lists what not to quote without its caveat.

- 1 Imagery: from COG to reflectance
- 2 Spectral indices
- 3 Smoothing: the daily curve
- 4 Soil armor: NDVI–DFI ternary unmixing
- 5 Phenology: planting and harvest
- 6 Off-season windows
- 7 Cover crop detection
- 8 Crop type: USDA CDL
- 9 Charts
- 10 Numbers not to quote without their caveat
- 11 External endpoints

September 2026 · Generated from METHODS.md · github.com/Field-to-Market/sandbox-remote-sensing-tool



1. Imagery: from COG to reflectance

`r/R/imagery.R`, `python/fieldrs/imagery.py`

Source and selection

Sentinel-2 L2A from Microsoft Planetary Computer's STAC API. Scenes are searched by the field's bounding box and filtered to `eo:cloud_cover < 70` (scene-level, `max_cloud`). Assets are read as Cloud-Optimized GeoTIFFs over HTTP range requests through GDAL's `/vsicurl/`, so only the window covering the field is transferred, not the scene.

How a COG read works, and which machine does it

Worth stating plainly, because it is usually assumed backwards: **the reading is done locally**. Microsoft serves bytes and nothing else.

A Cloud-Optimized GeoTIFF is an ordinary GeoTIFF written so it can be read in pieces. Measured on a Sentinel-2 B04 asset:

Full raster	10,980 × 10,980 px (≈230 MB uncompressed)
Internal layout	512 × 512 px tiles , not full-width strips
Compression	deflate, per tile
Embedded overviews	levels 2, 4, 8, 16

Tiling and the front-loaded index are the whole trick, and they are a property of how the file was *written*. That is the host's contribution. The other is an HTTP server that honors `Range` requests — Azure Blob Storage answers `bytes=1234-5678` with those bytes and has no idea it is serving geospatial data.

Everything else happens in GDAL, on the machine running this pipeline, through `terra` in R and `rasterio` in Python:

1. Range-request the first few kilobytes; read the TIFF header and tile index.
2. Reproject the field boundary into the raster's CRS.
3. Compute which 512 px tiles the boundary overlaps.
4. Range-request exactly those byte offsets.
5. Decompress and assemble the array.

On the 6.6 ha Georgia field that comes to a **20 × 36 px window, 0.0006% of the scene**. The reduction is not the host being clever on our behalf; it is GDAL declining to ask for the rest.

Three consequences follow, and they are why this matters beyond mechanism:

- **There is no server-side component.** COG works against dumb storage — Azure Blob, S3, a static Apache directory. Nothing needs to run at the far end.
- **This traffic is not API traffic.** Byte-range fetches against static files never touch the rate-limited STAC or token endpoints (§11), which is why the request volume there stays small however many scenes are read.
- **It is why the catalog is swappable** (§11). The reading machinery is ours and does not care whose bucket the bytes come from.

Band identifiers used:



Name here	Sentinel-2	Center	Native res.
blue	B02	490 nm	10 m
green	B03	560 nm	10 m
red	B04	665 nm	10 m
rededge1	B05	705 nm	20 m
nir	B08	842 nm	10 m
swir16	B11	1610 nm	20 m
swir22	B12	2190 nm	20 m
scl	SCL	—	20 m

coastal (B01, 60 m), rededge2 (B06), rededge3 (B07) and nir08 (B8A, all 20 m) are also mapped but unused by any index registered here.

20 m bands are resampled onto the 10 m grid. **SCL is resampled nearest neighbor**; a bilinear resample of a categorical mask invents classes that do not exist.

Scaling and the baseline offset

Stored DNs become bottom-of-atmosphere reflectance as

```
rho = (DN + offset) / 10000
```

offset is **0** for scenes processed under baseline < 04.00 and **-1000** for 04.00 and later (ESA changed the encoding in January 2022). The value is read per scene from the item's processing baseline rather than assumed. Skipping this shifts every index across the 2022 boundary and produces a step change that looks like a management event.

Cloud masking

Pixels are kept where SCL ∈ {4, 5, 6, 7} — vegetation, bare soil, water, unclassified. Everything else (cloud, cirrus, shadow, snow, saturated, defective) is set to NA.

Snow removal is deliberate and has a consequence worth stating: snowy dates disappear from the record rather than reading as bare soil, which is correct but thins winter coverage. Every off-season output therefore reports `n_obs` alongside the value.

Per-scene field statistic

For each scene and each index, pixels whose center falls inside the boundary are taken and the scene is either kept or dropped:

```
valid_frac = (clear field pixels) / (field pixels)
keep the scene if valid_frac >= min_valid      min_valid = 0.75
```

From the kept pixels the pipeline records `mean`, `median`, `sd`, `p10`, `p90`, `n`, `valid_frac` and the scene's cloud cover.

mean is what every downstream model uses.
Field to Market · Remote Sensing Methods · September 2026



Same-date de-duplication

Neighboring Sentinel-2 tiles overlap, so a field near a seam appears in two scenes on one date. Left alone these become duplicate x-values in the phenology spline and double-weight that date. One row per (index, date) is kept, ranked by `valid_frac` descending then `cloud` ascending.



2. Spectral indices

`r/R/indices.R`, `python/fieldrs/indices.py`. All are computed per pixel and then averaged over the field by §1, **not** computed from field-average reflectance — the two differ for any non-linear index.

Name	Formula	Display range
<code>ndvi</code>	$(\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$	-0.2 ... 1
<code>evi2</code>	$2.5 (\text{NIR} - \text{Red}) / (\text{NIR} + 2.4 \text{Red} + 1)$	-0.2 ... 1.2
<code>ndwi</code>	$(\text{Green} - \text{NIR}) / (\text{Green} + \text{NIR})$	-1 ... 1
<code>ndmi</code>	$(\text{NIR} - \text{SWIR1.6}) / (\text{NIR} + \text{SWIR1.6})$	-1 ... 1
<code>ndti</code>	$(\text{SWIR1.6} - \text{SWIR2.2}) / (\text{SWIR1.6} + \text{SWIR2.2})$	-0.2 ... 0.6
<code>ndi7</code>	$(\text{NIR} - \text{SWIR2.2}) / (\text{NIR} + \text{SWIR2.2})$	-1 ... 1
<code>ndsvi</code>	$(\text{SWIR1.6} - \text{Red}) / (\text{SWIR1.6} + \text{Red})$	-1 ... 1
<code>ndre</code>	$(\text{NIR} - \text{RedEdge1}) / (\text{NIR} + \text{RedEdge1})$	-0.2 ... 1
<code>savi</code>	$1.5 (\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red} + 0.5)$	-0.2 ... 1.2
<code>gcvi</code>	$(\text{NIR} / \text{Green}) - 1$	0 ... 12
<code>dfi</code>	$100 (1 - \text{SWIR2.2}/\text{SWIR1.6}) (\text{Red}/\text{NIR})$	-5 ... 45

`ndti`, `ndi7` and `ndsvi` are the multispectral residue indices evaluated by Sonmez & Slater (2016). `dfi` is the Dead Fuel Index of Cao et al. (2010).

CAI is not computable here. The hyperspectral cellulose absorption index the residue literature favors needs three narrow bands inside 2000–2200 nm; Sentinel-2 B12 is a single ~180 nm band covering the whole feature.

Two documented failure modes of NDTI

1. **Soil moisture moves it more than tillage does.** On the Iowa field, 18 and 20 May 2024 gave NDTI 0.035 and 0.138 with NDVI flat at 0.18 — a four-fold swing from rain with no change in residue. Single-date NDTI is not usable; the validated approach is multitemporal (minimum NDTI over a bare-soil window, Zheng et al. 2013).
2. **It does not separate residue from green cover.** NDTI reads 0.15–0.24 over a cover crop because green canopy absorbs in the SWIR. This is the defect that motivated §4.



3. Smoothing: the daily curve

```
daily_series() in r/R/extract.R; python/fieldrs/phenology.py
```

Observations are irregular because cloud decides when you get one. Threshold crossings need a continuous curve, so a penalized smoothing spline is fitted to (date, mean) and predicted onto a **1-day grid**.

- **R:** `stats::smooth.spline(x, y, spar = 0.35)`, and `spar = 0.5` inside the cover-crop module, which fits a shorter window.
- **Python:** `scipy.interpolate.make_smoothing_spline` at a **fixed penalty** `lam = 50.0`, not GCV-selected.

The fixed penalty is not arbitrary. Letting GCV choose per series put peak NDVI up to 0.081 above the highest actual observation in 41 of 77 seasons — a physically impossible curve — and disagreed with R on 22 seasons. At `lam = 50` the two implementations disagree on 3.

Gaps are not bridged. Any day further than `max_gap / 2 = 22.5` days from the nearest real observation is set to NA, so a two-month winter hole stays a hole instead of becoming a confident-looking line.

The spline is always fitted to the window in question plus a margin, never to the whole multi-year record: one smoothing parameter cannot represent six annual cycles, and fitting globally flattens the off-season features being measured.



4. Soil armor: NDVI–DFI ternary unmixing

r/R/armor.R, python/fieldrs/armor.py

Why it replaced minimum NDTI

NDTI measures crop residue, which is only half of soil cover: a field under a living cover crop or a perennial stand is protected just as well as one under stubble, and NDTI scores it as bare. On this library that error was not subtle — the Washington alfalfa pivot, the best-protected soil present, ranked **last of eleven**. It now ranks first.

The model

Each pixel is placed in (NDVI, DFI) space and decomposed into three fractions that sum to one:

- **fPV** — photosynthetic vegetation, living green cover
- **fNPV** — non-photosynthetic vegetation: residue, stubble, litter
- **fBS** — bare soil

with

$$DFI = 100 \left(1 - \frac{SWIR2.2}{SWIR1.6} \right) \left(\frac{Red}{NIR} \right)$$

The first term is the lignocellulose absorption near 2100 nm that residue has and soil does not; the second suppresses green canopy.

Solving the linear mixture means inverting

$$A = \begin{bmatrix} NDVI_{PV} & NDVI_{NPV} & NDVI_{BS} \\ DFI_{PV} & DFI_{NPV} & DFI_{BS} \\ 1 & 1 & 1 \end{bmatrix}, \quad \begin{bmatrix} f_{PV} \\ f_{NPV} \\ f_{BS} \end{bmatrix} = A^{-1} \begin{bmatrix} NDVI \\ DFI \\ 1 \end{bmatrix}$$

A^{-1} is computed once at load time and each fraction is an affine function of (NDVI, DFI), so the whole decomposition is three multiply-adds per pixel.

Endmembers

Fitted on **65,555 cloud-free pixels across six fields** spanning permanent green, heavy residue and clean tilled soil:

Endmember	NDVI	DFI
PV (living canopy)	0.961	1.212
NPV (residue)	0.232	25.642
BS (bare soil)	0.183	3.296

NPV and BS differ by only 0.05 in NDVI and by 22 in DFI — **DFI is what separates residue from soil; NDVI alone cannot.**

Water and deep shadow were excluded before fitting, and that mattered: as NIR collapses, DFI's Red/NIR term explodes, so the highest-DFI pixels in the raw pool were flooded rice and cloud shadow rather than residue. Left in, they put the NPV vertex at NDVI −0.02, which is not a surface any crop field has.



Clamping and masking

Spectral variability puts roughly **one pixel in eight** just outside the triangle. Those are clamped to [0, 1] and renormalized to sum to one, rather than discarded: a pixel at -0.03 is a pure endmember with noise on it, not a measurement failure.

Pixels below **NDVI 0.05** are water or deep shadow, where DFI is meaningless. They are masked to NA so they drop out of the field average instead of corrupting it.

```
armor = 1 - fBS
```

The summary statistic: time-weighted, not median or minimum

Over the reporting window, armor is summarized by a **trapezoidal average over observation dates**:

```
armor_bar = SUM[ (t_{i+1} - t_i) (a_i + a_{i+1}) / 2 ] / (t_n - t_1)
```

Minimum was right for NDTI, where a single wet morning could quadruple the reading. A cover fraction is a different quantity, so this started as a median — but the cross-implementation check caught that being fragile: on the Georgia field in 2026 the two implementations saw 7 and 8 observations of a steeply falling series and the extra date moved the median from 0.59 to 0.72. The trapezoidal average gave 0.500 against 0.497.

It is also the right quantity on its own terms: soil is exposed to erosion over *time*, so what matters is average cover through the window, not the middle value of however many scenes happened to be clear.

Extraction span vs reporting window — these are different

- `ARMOR_SPAN = 01-01 ... 12-31`. The imagery pass covers whole calendar years, so the charts show fall tillage, cover crop establishment and overwinter residue loss.
- **The reported figure still comes from the pre-planting window only**, from `residue_window()` (§6). Widening the extraction did not change any number the validation scored.

`ARMOR_MIN_OBS = 3`: fewer usable observations in the window and the season is reported as unreadable rather than summarized.

Provisional bands

`ARMOR_BREAKS = (0.40, 0.75)` → "mostly bare" / "partly covered" / "well covered". These are this library's own distribution, rounded. Validation suggests crop-specific values near 0.44 (cotton) and 0.59 (peanuts); they were deliberately **not** written into the source, because fitting a threshold on 226 Georgia fields and shipping it as a global constant is the mistake the whole document warns about.



5. Phenology: planting and harvest

r/R/phenology.R, python/fieldrs/phenology.py

Operates on the daily NDVI curve (§3) for one calendar year. Requires ≥ 10 observations and ≥ 60 non-NA days in the curve.

1. Peak. Maximum of the smoothed curve inside the peak window, **1 June – 15 September** by default. If `peak < 0.45`, the season is reported as "no summer crop detected" — fallow, or a failure.

2. Applicability. The annual-row-crop model is abandoned if

```
min(curve) >= 0.30    AND    fraction of year above NDVI 0.50 >= 0.80
```

Both conditions, not either. A threshold on the minimum alone does not work: a heavily cover-cropped Maryland corn field bottoms out at 0.31 and the alfalfa pivot at 0.33, which is not a gap you can put a threshold in. What separates them is time-above: alfalfa years sit above 0.50 for 86–95% of the year, every genuine row-crop year at or below 72%.

3. Baseline. Within 1 March → peak – 20 days, take `gmin = min(curve)` and then the **last** day satisfying `value <= gmin + 0.06`.

Taking the last near-minimum rather than the global one is deliberate. On a cover-cropped field the curve dips twice before the cash crop: winter dormancy, then cover-crop termination. Anchoring on the global minimum reads the spring cover-crop green-up as crop emergence — which produced a February planting date for corn before it was fixed.

4. Green-up. With `amp = peak - baseline` (abandon if `amp < 0.2`), the first day on the rising limb where

```
value >= baseline + 0.20 * amp
```

5. Planting = green-up – crop lag. Lags are days from planting to 20% of amplitude, **literature-typical and uncalibrated**:

Crop	Lag (days)
Corn	21
Soybeans	18
Winter Wheat	0
Cotton	24
Sorghum	20
Rice	20
default	20

Listed in the order they appear in `CROP_LAGS`, so the table can be checked against the source line by line. **Winter wheat is zero because it is planted the previous autumn** — the spring rise is regrowth from an established stand, not emergence, so there is no lag to subtract. `default` applies to any crop CDL reports that is not named above.

6. Harvest. Anchored on the **post-peak minimum**, not the spring baseline. Requires a real drop, `peak - post_trough >= 0.35 * amp`, then the first day at or below `post_trough + 0.20 (peak - post_trough)`.

Spring anchoring fails whenever something green follows the cash crop: on a cover-cropped cotton field NDVI falls after



for five seasons running. The amplitude guard stops a season still under way being read as an early harvest.

7. Uncertainty from actual observation density. Rather than quoting a flat ± 14 days, the window is widened by the real gap in observations around each crossing:

```
window = max(14, ceil(nearest observation gap))
```

Confidence is **high** if the larger gap ≤ 10 d, **medium** if ≤ 20 d, otherwise **low**. A crossing sitting in the middle of a three-week cloud gap is a guess and says so.

Untested. No planting or harvest estimate in this pipeline has ever been compared against a grower-reported date. This is the largest uncalibrated component.



6. Off-season windows

`residue_window()` in `r/R/phenology.R`

The bare-soil period between one crop's harvest and the next crop's planting:

```
start = harvest(year-1) + 7      RESIDUE_HARVEST_LAG
end   = planting(year)  - 3      RESIDUE_PLANT_LEAD
```

Either end falls back to fixed dates when phenology could not supply it — `start = Oct 15 (year-1)`, `end = May 10 (year)` — the same fallbacks the cover-crop module uses, so the two agree on what "the off-season" means. The `spring` season additionally clips `start` to 1 March. A window whose end is at or before its start returns nothing rather than a negative span.

The 7-day lag lets residue settle after harvest traffic; the 3-day lead stops before planting disturbs the surface.



7. Cover crop detection

`r/R/covercrop.R`, `python/fieldrs/covercrop.py`

Window Oct 15 (fall year) → May 10 (following year), narrowed to `harvest + 7 ... planting - 3` when phenology supplies them.

Constant	Value	Meaning
<code>CC_RESIDUE_MAX</code>	0.25	NDVI ceiling for crop residue
<code>CC_GREEN_MIN</code>	0.30	level sustained green must reach
<code>CC_STRONG</code>	0.35	level for a confident call
<code>CC_WATER_NDVI</code>	0.05	open water
<code>CC_WATER_SHARE</code>	0.25	share of window flooded before "not detectable"

`green_days` is the count of days in the window where the smoothed curve (`spar = 0.5`, fitted to the window ± 45 days) is at or above `CC_GREEN_MIN`.

Verdicts, in order of precedence: winter cash crop (from CDL) → double crop → `likely cover crop` (`max` ≥ 0.35 and ≥ 2 observations above 0.30) → `possible cover crop` (`max` ≥ 0.30) → `no cover crop detected`. Perennial stands and persistently flooded windows return "not applicable" and "not detectable" respectively — which are different answers from "none".

These thresholds are measured wrong outside the Corn Belt. Against 347 Georgia field-winters the shipped detector said "likely cover crop" to 332 of 347 fields: **98.4% sensitivity, 7.1% specificity, 57.5% accuracy** against a 55.2% majority-class baseline. The cause is climate, not code — 98.1% of *no-cover* Georgia fields exceed NDVI 0.30, and the median one peaks at 0.53. The residue ceiling the detector is built around does not exist there. Refit per region before surfacing the verdict.



8. Crop type: USDA CDL

`r/R/cropland.R`, `r/R/cdl_local.R`, `python/fieldrs/cropland.py`

30 m annual crop-type raster, CONUS, EPSG:5070 (Albers equal-area). Read from a local bulk copy when present, falling back to the CropScape API.

Clipping uses center-in-polygon (`touches = FALSE`). The default — every cell the polygon so much as clips — returned 233 cells (21.0 ha) on a 17.2 ha field, **22% too many**, and those extra cells carry the neighboring field's crop. That inflates every CDL area figure and biases the boundary check toward "split".

Local and API agree on the dominant crop for **346 of 347** field-years; the one disagreement is 33.0% Corn against 32.7% Peanuts. Against polygon geometric area the API is +0.06% mean and local +0.23%, both inside 30 m pixel quantization.

Boundary check (`cdl_split_advice`)

Per year, the dominant class and its share. Then, on the value matrix across years:

```
dom_k    = modal class of year k
dis_ik   = 1 if pixel i differs from dom_k in year k, else 0
persist_i = SUM_k dis_ik / (years pixel i was observed)
flagged_i = persist_i >= 0.6                                SPLIT_PERSIST
frac      = flagged pixels / field pixels
```

Verdict: **split** if `frac >= 0.12` (`SPLIT_MIN_FRAC`) and the largest flagged block `>= 2.0 ha` (`SPLIT_MIN_HA`); **trim** instead if $\geq 60\%$ of the flagged area is non-crop (farmstead, road, water, woodland); otherwise **single unit**.

This is computed on the extracted value matrix rather than with `terra::app()` across layers — the per-layer rasters come back from `app()` without values materialised, and stacking them yields an all-NA result that silently reports no disagreement at all, however split the boundary actually is.

Measured: boundary purity predicts crop agreement almost completely. Under 60% single-crop, CDL matched the grower **12.9%** of the time; above 75%, **88.7%**. Gating at $\geq 75\%$ keeps 84% of rows and lifts overall agreement from 80.7% to 88.7%.

Hand-drawn boundaries

`r/R/draw.R`. A traced polygon is validated (`st_make_valid`), area-checked (0.5–500 ha) and CONUS-checked, then its CDL history is read and shown *before* any imagery is fetched. Purity bands in the verdict are the validation numbers above.



9. Charts

Every figure is base R graphics. No ggplot2 anywhere.

Percentile stretch (all RGB and false-color imagery)

`stretch_band()` in `r/R/plotting.R`:

```
lim = quantile(v, c(0.02, 0.98))
out = clamp((v - lim1) / (lim2 - lim1), 0, 1)
out = clamp(out^0.8 * 255, 0, 255)
```

The 2–98% clip discards outliers that would otherwise flatten the image; gamma 0.8 lifts the mid-tones. **The stretch is per scene**, so brightness is not comparable between panels of a contact sheet — only structure is.

Composite	Channels
True color	Red, Green, Blue
False color	NIR, Red, Green

Index maps

`plot_index()` — a fixed color ramp over the index's registered display range, so the same color means the same value in every panel and across fields. Soil armor uses the vegetation palette over 0–1.

Soil armor, per calendar year (`plot_armor`)

One panel per year, up to 4 columns. Within each panel:

- x: 1 Jan → 31 Dec, ticks every 2 months (twelve labels collide at quarter width)
- y: cover fraction, fixed 0–1 so panels are comparable
- **filled teal** = fPV from 0; **filled tan** = fNPV stacked on top; the **unfilled white gap to the top is fBS** — the quantity being measured is the empty space, not a color needing a legend
- a gray line and points at fPV + fNPV = armor
- **gray shaded band + dashed lines** = the pre-planting window. The panel title quotes the armor figure, which is averaged over that band only, so the shading stops the eye reading an annual average off a twelve-month chart
- legend on the first panel only

Areas are drawn between consecutive observations. Sparse stretches therefore show as long straight segments — honest about where there is no data, but not to be read as measurement.

CDL matrix (`plot_cdl_matrix`)

One small map per year, row-major so years read left to right then down, with one shared legend in an extra row. **Colors are assigned once across all years**, so a crop keeps its color from panel to panel; classes under `min_pct = 1` are dropped from the legend. A boundary that is one management unit shows one color filling it every year — that visual is the whole diagnostic.



Boundary disagreement (`plot_cdl_split`)

Maps `persist_i` — the share of years each pixel differed from that year's dominant class. Pale means it agrees every year; a persistent block outlined in red is a sub-area farmed separately. One red year is classification noise, five is a second field.

Cover crop matrix (`plot_cover_crop_matrix`)

Every winter on a shared Oct → Jun x-axis and a common y-axis, one panel per winter, so winters stack comparably. Threshold lines at `CC_GREEN_MIN` and the residue ceiling.

Phenology (`plot_phenology` , `plot_phenology_timeline`)

Observations as points, the smoothed daily curve as a line, and vertical markers at estimated planting and harvest with their uncertainty windows shaded. The timeline variant stacks seasons on a common day-of-year axis.

Data availability (`plot_catalog_timeline`)

One row per source (Sentinel-2, Landsat 8/9, HLS, Sentinel-1, MODIS, NAIP ...), one mark per acquisition. Shows what exists, not what is usable — cloud is not applied here.

Contact sheets (`plot_scene_grid`)

Every scene in a quarter as thumbnails at `max_px = 180` , cached per (field, scene set, view). A dimensionality guard refuses to cache a 3-band array under a single-index key — a bug that once poisoned the cache with RGB arrays stored as if they were NDVI, and then masked its own fix.



10. Numbers not to quote without their caveat

Number	What it is	Caveat
ARMOR_ENDMEMBERS	6 fields, 4 states	Looks empirical, is local. Refit before use elsewhere
ARMOR_BREAKS 0.40 / 0.75	This library's distribution	Validation suggests ~0.44 / 0.59, crop-specific
RESIDUE_BREAKS 0.05 / 0.09	p33 / p75 of 64 seasons here	Rank among eleven fields, not an absolute scale
CC_GREEN_MIN 0.30, CC_STRONG 0.35	Corn Belt literature	Measured wrong for Georgia — 7.1% specificity
CROP_LAGS	Literature-typical	Never calibrated against a planting date
SPLIT_*	Chosen	Never calibrated, though the purity bands behind them now are

Soil armor is a **cover** metric, not a tillage metric. Validation showed its apparent tillage signal running largely through cover-crop adoption: a conventionally tilled cotton field *with* a cover crop reads 0.604, statistically indistinguishable from a conservation-tilled one at 0.625 and far above a conventionally tilled field without one at 0.344.

11. External endpoints

Every network call the pipeline makes. Listed here so a reader can see exactly what the numbers in this document were derived from, and reproduce them.

Server-side — requested by the R or Python process:

Endpoint	Method	Used by
<code>planetarycomputer.microsoft.com/api/stac/v1</code>	POST	<code>search_scenes()</code> , <code>field_catalog()</code>
<code>planetarycomputer.microsoft.com/api/sas/v1/token/{collection}</code>	GET	Asset signing, called by <code>rstac/planetary_computer</code> rather than by this code
<code>sentinel2l2a01.blob.core.windows.net/sentinel2-l2a/...</code>	GET (range)	Every COG read, through GDAL <code>/vsicurl/</code>
<code>nassgeodata.gmu.edu/axis2/services/CDLService/GetCDLFile</code>	GET	<code>get_cdl()</code> , fallback only once a local CDL store exists (§8)
<code>www.nass.usda.gov/.../datasets/{year}_30m_cdls.zip</code>	GET	<code>download_cdl.py</code> , once per year per machine

Browser-side — requested by the viewer's browser, never by the server:

Endpoint	Used by
<code>server.arcgisonline.com/.../World_Imagery/MapServer/tile/{z}/{y}/{x}</code>	Satellite basemap
<code>server.arcgisonline.com/.../Reference/World_Boundaries_and_Places/...</code>	State lines and place names
<code>server.arcgisonline.com/.../Reference/World_Transportation/...</code>	Major roads

Seven STAC collections are queried through the single STAC endpoint: `sentinel-2-l2a`, `landsat-c2-l2`, `hls2-s30`, `hls2-l30`, `sentinel-1-rtc`, `modis-13Q1-061` and `naip`. **Only `sentinel-2-l2a` is read for analysis** — every index, fraction and date in this document comes from it. The other six are counted, never read, to report what else exists over a field.

No account, key or token is needed for any of them. The SAS token is fetched anonymously and lasts about 45 minutes; `rstac` caches it per container, so signing several hundred scenes costs one request rather than several hundred.

The catalog is swappable; the data is not

Worth separating, because the two are often conflated. **Sentinel-2 is an ESA mission** — the European Space Agency builds, flies and processes it, and publishes the imagery free and open under Copernicus. **Planetary Computer is Microsoft's mirror of it**: a copy on Azure Blob Storage with a STAC catalog in front. Microsoft neither produces nor owns the data.

So the imagery is a hard dependency and the catalog is not. The same scenes are mirrored elsewhere, and the closest alternative is `earth-search.aws.element84.com/v1` on AWS. Checked against it directly:



	Planetary Computer	Earth Search (AWS)
Collection id	<code>sentinel-2-l2a</code>	<code>sentinel-2-l2a</code> — identical
Asset keys	<code>B04</code> , <code>B08</code> , <code>B11</code> , <code>B12</code> , <code>SCL</code>	<code>red</code> , <code>nir</code> , <code>swir16</code> , <code>swir22</code> , <code>scl</code>
Asset host	Azure Blob Storage	<code>sentinel-cogs.s3.us-west-2.amazonaws.com</code>
Signing	SAS token, ~45 min	none — public S3

Moving would be *simpler* than the present arrangement, not harder. The collection id is the same string; the signing step disappears; and Earth Search's asset keys are already the internal band names this pipeline uses (§1), because both follow the STAC common-band convention while Planetary Computer exposes ESA's raw B-numbers. `S2_BANDS` would become an identity map.

Nothing in this document would change. Every formula, endmember, threshold and validated figure describes Sentinel-2 surface reflectance, not the route it arrived by. What would change is `PC_STAC`, the band map and the signing call — which is the argument for keeping those three behind one seam when this is ported, rather than spread through the read path.

One correction, since it is the obvious thing to reach for: ESA's own **Copernicus Data Space is not a drop-in STAC swap**. Its `/stac/collections` endpoint lists ten collections and no Sentinel-2 among them; its Sentinel holdings are served through different interfaces. It is the authoritative source, not an equivalent endpoint.

Request volume, measured

Call	Volume	Goes to
STAC search	1 per unique field and date range, then cached on disk	Rate-limited API
SAS token	~1 per 45 minutes	Rate-limited API
COG range reads	thousands per field	Azure Blob Storage

Nearly all of it is the third row, which is bulk object storage rather than an API. Searches retry four times at 2/6/15 s and honor `Retry-After` when the service sends one; the throttling seen in practice came from four heavy searches in quick succession, not from sustained volume.



References

- Cao, X., Chen, J., Matsushita, B., Imura, H. (2010). Developing a MODIS-based index to discriminate dead fuel from photosynthetic vegetation. *International Journal of Remote Sensing*. — DFI
- Sonmez, N. K., Slater, B. (2016). Measuring intensity of tillage and plant residue cover using remote sensing. *European Journal of Remote Sensing*. — NDTI, NDI7, NDSVI, CAI
- Zheng, B., Campbell, J. B., de Beurs, K. M. (2013). Remote sensing of crop residue cover using multi-temporal Landsat imagery. *Remote Sensing of Environment*. — minimum-NDTI method
- USDA NASS Cropland Data Layer — <https://nassgeodata.gmu.edu/CropScape/>
- Microsoft Planetary Computer, Sentinel-2 L2A — <https://planetarycomputer.microsoft.com/dataset/sentinel-2-l2a>

